23CSE455 Design Patterns
Case Study Report

Design Pattern Identification and Evolution in a Spring MVC-Based Warehouse Management Application

Team: B9
Date: 10 October 2026 (findings re-audited 11 October 2026)
Submission repository: 23CSE455_DP_CaseStudy
Team working repository: akdswordguy/Team_B9_DP

Roll number	Student name	Batch
AM.SC.U4CSE23215	D Aditya Kiran	CSE C
AM.SC.U4CSE23235	Mithun Krishna	CSE C
AM.SC.U4CSE23248	Sai Kishen K M	CSE C
AM.SC.U4CSE23359	Sivadev Smijith	CSE D

Per-part findings documents with full evidence tables accompany this report: `report/part1_findings.pdf`, `report/part2_findings.pdf`, and `report/part3_findings.pdf`. Unless stated otherwise, evidence paths below are relative to `Aditya_findings/` in the team working repository.

1 Application and architecture

1.1 Original source and baseline

The application studied is OpenWMS, an open-source warehouse-management system built from Java/Spring Boot components. OpenWMS is split across several repositories; three were inspected, each at an exact baseline commit:

Repository	Role in this study	Baseline commit
`openwms/org.openwms` (CORE)	Part 1 pattern inspection; contains the `org.openwms.core.util` library	`9d263bf65ec75301e0302539ae7bf581412742b8`
`openwms/org.openwms.common.service.lib`	Multiple-Warehouse Support (Part 2)	`a6566d7a90364f8cb0e82ec013720b644926dffe`
`openwms/org.openwms.common.service`	Multiple-Warehouse Support (Part 2)	`fc7edd4ffbe16de65d482b3a2d462d4d896b1998`

The CORE repository was clean at inspection. **Multiple-Warehouse Support** changes in the two Common Service worktrees were not committed at the time of the recorded audit.

OpenWMS repositories inspected in this work are distributed under the Apache License 2.0. Their copyright, licence, provenance, and exact baseline commits must be preserved when republishing patches or source excerpts. The **Barcode-Based Picking** and **Automatic Stock Reordering** projects and the Part 3 implementations are separate case-study reference projects, not modifications to the canonical Picking and Inventory modules, which were inaccessible (Section 1.3).

1.2 Environment

The inspected application and evolution projects use Java and Maven. **Barcode-Based Picking** and **Automatic Stock Reordering** verification was performed with Maven and Java in the local Windows environment. Part 3 toolchains recorded in `PART3_EVIDENCE.md` were OpenJDK 25.0.4.1, Python 3.13.15, Node.js 24.21.0, and g++ 16.2.0; the 11 October re-audit re-ran Part 3 with OpenJDK 21.0.12.1, Python 3.13.16, Node.js 22.22.0, and g++ 13.3.0. Static complexity was measured with Lizard 1.24.1. Exact dependency versions for each upstream module should be taken from its checked-in Maven descriptors and are not inferred here.

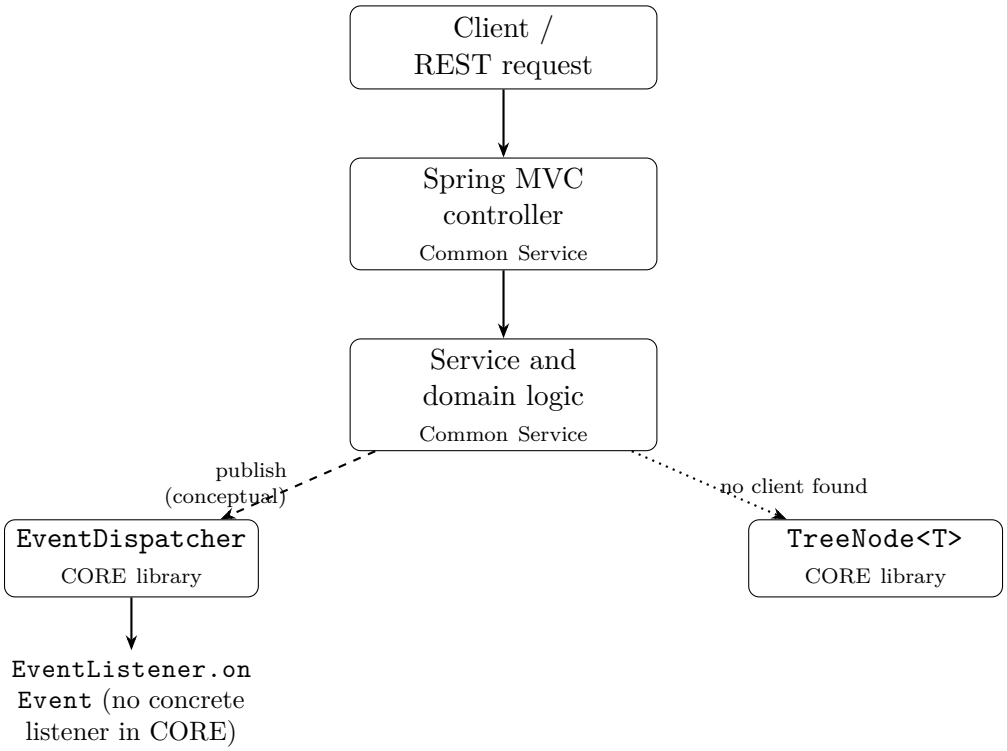
1.3 Scope and exclusions

Part 1 examined the `org.openwms.core.util` module of the CORE repository (64 Java files at the baseline) for two candidate GoF collaborations: the event-dispatching collaboration involving `EventBroker`, `EventDispatcher`, `EventListener`, `SimpleEventDispatcher`, and `RootApplicationEvent`; and the generic tree utility `TreeNode<T>` / `TreeNodeImpl<T>`. Three further non-GoF mechanisms were recorded separately (Section 2.6). Candidate patterns were accepted only where source relationships and behavior supported the pattern interpretation. Singleton and Proxy were assessed and rejected. The repository contains no concrete client of the generic Composite utility and no concrete implementation of `EventListener`; both limitations are retained in the findings.

Multiple-Warehouse Support modifies the Common Service Library and Common Service worktrees to model multiple warehouses. **Barcode-Based Picking** and **Automatic Stock Reordering** are isolated reference implementations because the canonical modules could not be modified: the CORE README lists the Picking Library as a private, GPLv3-licensed repository and the Inventory Service as private preview. Part 3 contains small independent implementations of Observer and Composite in four languages; they are not claimed to be OpenWMS production code.

1.4 Architecture and request/event flow

OpenWMS uses a microservice architecture in which each business component is a Spring Boot service in its own repository. The CORE repository is not itself a service: its README states that it holds project-level documentation and one Maven module, `org.openwms.core.util`, a library of utilities, exception types, and shared event types. The Spring MVC controllers and service logic referred to in the title live in the service repositories such as Common Service. The diagram below is a scope diagram, not a claim that every shown module is deployed in one process or that the Composite utility is used by a specific business workflow.



Event flow. A producer calls `EventPublisher.publish`; `NonBlockingEventPublisherImpl` forwards the event asynchronously (Spring `@Async`) to `EventDispatcher.dispatch`, and `Simple EventDispatcher` synchronously calls `onEvent` on each `EventListener` subscribed to that event's exact class. No concrete producer or listener call was traced in the inspected source. The generic tree utility supports parent/child composition and navigation, but no warehouse business workflow in the inspected source uses it.

1.5 Run instructions and recorded results

For the upstream modules, use the module README and Maven descriptors for setup. **Multiple-Warehouse Support** verification was recorded against the existing Common Service Library and Common Service worktrees; the exact baseline and verification addendum are in `LLM_RECORDS/LLM_RECORDS.md`. **Barcode-Based Picking** and **Automatic Stock Reordering** can be run independently from their project roots with Maven:

- **Barcode-Based Picking**: `mvn.cmd-DforkCount=0verify` — recorded `BUILD SUCCESS`, 14/14 tests.
- **Automatic Stock Reordering**: `mvn.cmd-DforkCount=0verify` — recorded `BUILD SUCCESS`, 19/19 tests.

The `-DforkCount=0` option runs tests in the Maven process and may affect test isolation compared with forked execution. Maven could not be re-run during the re-audit because Maven Central was blocked in the audit environment, so these totals are the team's recorded runs; the source-level test counts were re-checked and match. For Part 3, use the per-language commands documented in `Part3-cross-language/PART3_EVIDENCE.md`; all eight implementations passed their recorded harness checks and passed again in the re-audit. The original project does not have a single verified, reproducible one-command build documented by this case study.

2 Part 1 Pattern discovery

2.1 Method

Candidates were assessed through source-level role and relationship mapping, not by names alone. For each candidate, the inspection considered intent, collaborating types, method signatures and bodies (registration, notification, delegation, child management), and available behavior. Every line citation was re-checked against the upstream baseline in the re-audit and found exact, and the copied files under `Part1_Findings/evidence/source/` are byte-identical to upstream. The evidence bundle is `Part1_Findings/Part1_Findings.md`, `Part1_Findings/data/patterns.csv`, `Part1_Findings/data/rejected_candidates.csv`, `Part1_Findings/data/pattern_counts.csv`, `Part1_Findings/evidence/verification_notes.md`, and the PlantUML sources with rendered PNGs in `Part1_Findings/diagrams/`.

2.2 Totals at the baseline

Measure	Count	Interpretation
Confirmed GoF pattern types	2	Observer and Composite.
Confirmed pattern instances	2	One event-dispatching Observer collaboration and one generic tree Composite collaboration.
Concrete participant classes	2	`SimpleEventDispatcher` and `TreeNodeImpl<T>`.
Unique participant symbols	7	Five in the Observer mapping and two in the Composite mapping; no symbol is shared.
Rejected candidates assessed	2	Singleton and Proxy were assessed and rejected for insufficient supporting evidence.
Uncertain candidates	0 recorded	No separate uncertain candidate is counted in the frozen inventory; this does not mean all possible patterns in the whole product were exhaustively ruled out.

The scope is the accessible source examined, not a claim that the entire OpenWMS ecosystem contains only two design patterns.

2.3 Pattern inventory

Pattern type	Instance name	Instances	Concrete classes
Observer	OpenWMS event subscription and dispatch	1	1
Composite	Generic `TreeNode` hierarchy	1	1

2.4 Instance 1: OpenWMS event subscription and dispatch

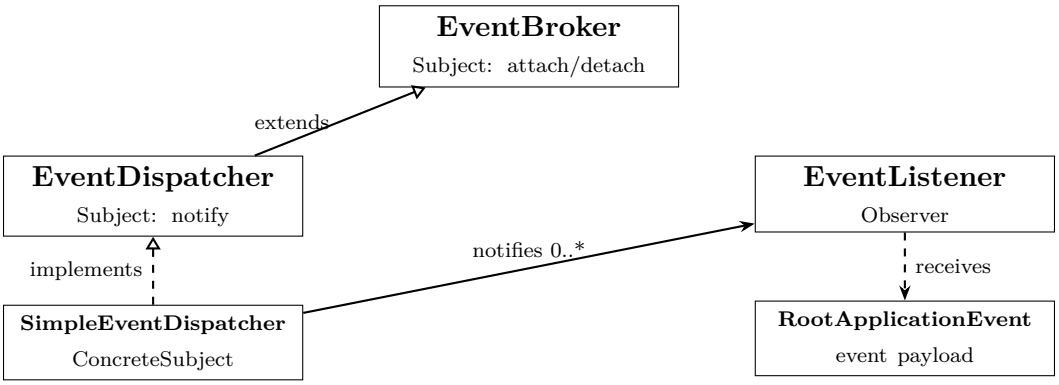
Pattern and category. Observer — GoF behavioral pattern.

Purpose and origin. The collaboration decouples code that raises application events from the listeners that react to them, through explicit subscription, unsubscription, and dispatch. It is part of the inspected OpenWMS source, rather than a pattern introduced by this case study.

Participant mapping (paths relative to `org.openwms.core.util/src/main/java/org/openwms/core/`)

Pattern role	Actual symbol	Evidence location
Subject: attach/detach	`EventBroker`	`event/EventBroker.java:19-56`; `subscribe`/`unsubscribe` by listener or bean name.
Subject: notify	`EventDispatcher`	`event/EventDispatcher.java:19-34`; extends `EventBroker`, adds `dispatch`.
ConcreteSubject	`SimpleEventDispatcher`	`event/SimpleEventDispatcher.java:27-38,54-56,75-79,99-106`.
Observer	`EventListener`	`event/EventListener.java:19-31`; `onEvent(RootApplicationEvent)`.
Event payload	`RootApplicationEvent`	`event/RootApplicationEvent.java:23-29`; extends Spring `ApplicationEvent`.
ConcreteObserver	—	None: no class in the repository implements `EventListener`.

Evidence and verification. The defining Observer behavior is present: listeners attach and detach at runtime, and `dispatch` iterates the registered set and invokes the common update method `onEvent`; the subject never refers to a concrete listener type. The subject role is split across two interfaces and one class rather than invented as a single `Subject` type. `NonBlockingEventPublisherImpl` calls `dispatch` asynchronously; in GoF terms it is a client that triggers notification, so it is recorded as the separate mechanism N01 rather than counted as a participant. The diagram below is a conceptual role diagram; `Part1_Findings/diagrams/observer_uml.puml` is the editable source.



Observations and limitations. (i) `dispatch` looks up listeners by `event.getClass()`, so a listener subscribed to `RootApplicationEvent` is not notified of a subclass event such as `UserChangedEvent`. (ii) `SimpleEventDispatcher` notifies subscribers sequentially and synchronously. (iii) With no ConcreteObserver in scope, the instance is confirmed at library level.

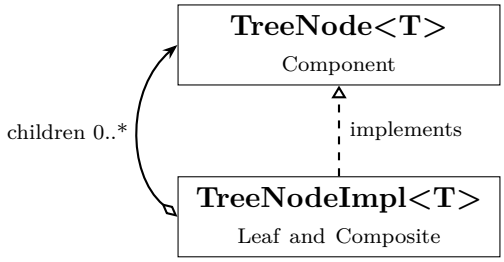
2.5 Instance 2: Generic tree hierarchy

Pattern and category. Composite — GoF structural pattern (transparent variant, single implementation class).

Purpose and origin. `TreeNode<T>` defines a node holding a data value and an identifier-keyed collection of child nodes of the same type, so hierarchies of any depth can be built and navigated through one abstraction. It is a utility-level collaboration present in OpenWMS source.

Participant mapping

Pattern role	Actual symbol	Evidence location
Component	`TreeNode<T>`	`util/TreeNode.java:23-115`; data, `isLeaf`, `getChildren`, `getChild`, `addChild`, `removeChild`, parent access.
Leaf and Composite	`TreeNodeImpl<T>`	`util/TreeNodeImpl.java: 29-33,47-57,64-68,83-108`; `LinkedHashMap` of children; `isLeaf()` = `childrenMap.isEmpty()`.
Client	—	None: no class in the repository uses `TreeNode` or `TreeNodeImpl`.



Children are `TreeNode<T>`, giving a recursive structure; traversal is left to the client.

Verification and limitation. Two Composite properties are present: part–whole recursion (children are themselves `TreeNode<T>`) and uniform treatment (child management is declared on the component interface, the GoF transparent form). There are no separate Leaf and Composite classes; "leaf" is a runtime state. `TreeNode` declares no recursive component operation, so the classification rests on structure and interface rather than recursive behavior. No concrete application/domain client using `TreeNode` or `TreeNodeImpl` was found, so the result is reported as a confirmed generic utility-level pattern instance, not evidence that a particular warehouse workflow uses Composite.

2.6 Non-GoF mechanisms and rejected candidates

Three further mechanisms were recorded separately from the GoF totals:

ID	Mechanism	Evidence	Status
N01	Asynchronous publish/subscribe	`EventPublisher.java:19-33`; `NonBlockingEventPublisherImpl.java:22-50` (`@Asyncpublish` delegates to `dispatch`).	Confirmed
N02	Spring application-event listener	`LocalServiceInitializer.java: 27-33,50-53` (`ApplicationListener<ContextRefreshedEvent>`).	Confirmed
N03	Fire-after-transaction AOP advice	`FireAfterTransactionAspect.java: 30-36,52-66,76-107`. The class is a plain `@Component`; no `@Aspect`, pointcut, XML `aop:` config or AspectJ dependency exists in the repository.	Partially confirmed (wiring not in scope)

Candidate	Reason for rejection
Singleton	Spring `@Component` scope alone does not establish GoF Singleton. The candidate beans have public constructors and no static instance or accessor; they are single only because the container scopes them that way.
Proxy	Framework-generated proxies/AOP behavior is not counted as an application-level GoF Proxy, and no application class controls access to a separate subject through the same interface.

The rejected candidates and reasons are preserved in `Part1_Findings/data/rejected_candidates.csv`. These are scope-bounded rejections, not proof that the patterns occur nowhere in the larger ecosystem.

3 Part 2 Evolution and refactoring

The three changes are evaluated against their recorded acceptance criteria, commits, tests, and metrics. **Multiple-Warehouse Support** is an uncommitted modification to the existing OpenWMS Common Service worktrees. **Barcode-Based Picking** and **Automatic Stock Reordering** are separate reference applications, because the canonical Picking and Inventory modules were inaccessible. They must not be presented as upstream merged changes.

3.1 Multiple-Warehouse Support — Multiple warehouses

Change and acceptance criteria. Introduce explicit warehouse identity so that locations with identical coordinate keys can exist in different warehouses without collision. A location belongs to one warehouse; location identity is the combination of warehouse and the existing five-part `LocationPK`. Coordinate uniqueness is warehouse-scoped, while PLC/ERP identifiers remain globally unique. `LocationGroup` remains a logical/hierarchical grouping rather than becoming a warehouse. Unqualified lookups that are ambiguous across warehouses must not silently choose one.

Acceptance criteria and verification

Criterion	Expected result	Verification evidence	Status
Same coordinates in different warehouses	Both locations coexist, distinguished by warehouse.	`LocationTest.same_coordinates_in_different_warehouses_are_distinct_locations`; `LocationIT.shall_allow_same_coordinates_in_different_warehouses`.	Recorded as passed
Duplicate coordinates in one warehouse	Rejected by the warehouse-scoped uniqueness rule.	`UC_LOC_ID` constraint on (`C_WAREHOUSE`, `C_AREA`, `C_AISLE`, `C_X`, `C_Y`, `C_Z`); no test exercises the rejection.	Enforced by schema; not tested
Warehouse-qualified lookup	Resolves the location within that warehouse.	`LocationServiceImplIT.shall_allow_same_location_id_in_different_warehouses_and_find_by_warehouse`; `TransportUnitServiceImplIT.update_with_warehouse_code_selects_the_requested_warehouse_for_duplicate_coordinates`.	Recorded as passed
Ambiguous unqualified lookup	Not silently resolved.	`TransportUnitServiceImplIT.update_without_warehouse_code_rejects_an_ambiguous_coordinate`; `TransportUnitCommandHandlerIT.rejects_unqualified_creation_when_the_coordinate_is_ambiguous`.	Recorded as passed
PLC/ERP identifier uniqueness	Globally unique across warehouses.	`UC_LOC_PLC_CODE` and `UC_LOC_ERP_CODE`, unchanged from the baseline; no dedicated test.	Preserved by schema; not tested
LocationGroup semantics	Stays a logical hierarchy, not a warehouse.	Source/design review.	Preserved by design
Legacy constructor compatibility	Legacy `LocationMO` constructor still works.	`LocationMOTest.legacy_nine_argument_constructor_preserves_null_warehouse`.	Recorded as passed
Production data rollout	Existing locations assigned to warehouses before rollout.	No migration/backfill supplied.	Not completed

Status note: the CH01 tests are identifiable by name in the published snapshot: 18 test methods were added (245 `@Test` annotations at the baseline, 263 after). The full list is in `report/part2_findings.pdf` and `Part2/CH01-multiple-warehouses/PROVENANCE.md`. The production rollout criterion is explicitly outstanding.

Before and after. The Common Service Library baseline is `a6566d7a90364f8cb0e82ec013720b644926dffe`; the Common Service baseline is `fc7edd4ffbe16de65d482b3a2d462d4d896b1998`. **Multiple-Warehouse Support** changes exist as uncommitted worktree changes in the two Common Service repositories; therefore there is no after commit ID. The re-audit re-derived the changed files by diffing the published snapshot against fresh upstream clones: 31 modified

and 13 new files in the Library and 2 modified files in Common Service, exactly as recorded in `PROVENANCE.md`. No upstream commit or merge is claimed.

Implementation. The change adds a `Warehouse` entity (with controller, service, repository, mapper and API types) and a mandatory warehouse reference on `Location`, adds `C_WAREHOUSE` to the `UC_LOC_ID` uniqueness constraint, and adds warehouse-qualified behavior where needed. The existing coordinate key remains five-part. No automatic default warehouse or speculative data migration was added.

Tests and recorded results. The verification addendum records a direct Maven installation run for the Common Service Library with `-DforkCount=0`: 187 tests, zero failures/errors, and 7 skipped; the build reported `BUILD SUCCESS`, although Asciidoctor emitted errors in the build log. Common Service verification reported `BUILD SUCCESS`, 1 test, zero failures/errors/skips; Failsafe reported no integration tests. Because the fork configuration can reduce isolation and the documentation plugin emitted errors, these results should be interpreted with those caveats. See the dated addendum in `LLM_RECORDS/LLM_RECORDS.md`.

Pattern effects. No new GoF pattern was confirmed. **Multiple-Warehouse Support** extends the domain model and warehouse-qualified lookup behavior; the Part 1 Observer and Composite instance names remain unchanged. It is not evidence that the generic Composite utility became a domain-level warehouse hierarchy.

Refactoring and behavior preservation. The main structural change is replacing location-only identity assumptions with warehouse-qualified identity while preserving the existing coordinate key. The behavior-preservation cases are same coordinates in separate warehouses, ambiguity for unqualified lookups, warehouse-qualified transport-unit operations, and compatibility of the legacy `LocationMO` constructor; the named tests above cover them. A production rollout still requires an operator-supplied mapping/backfill for existing locations, because the new warehouse column is non-nullable.

Before/after metrics. No **Multiple-Warehouse Support** before/after LOC metrics were recorded. They are not estimated in this report. The after changes are not committed, so a stable after-commit measurement cannot be referenced.

Limitations. No Flyway/Liquibase migration or production backfill is included; ORM auto-update is not a substitute for a deployment-specific data migration plan. Existing production locations require an explicit warehouse assignment before rollout.

3.2 Barcode-Based Picking — Barcode-based picking

Change and acceptance criteria. Add `POST/api/picking/barcode` to resolve a scanned barcode to stock and process a pick. A successful pick returns success; malformed/invalid requests return 400, unknown barcodes return 404, and unavailable or insufficient stock returns 409. The pre-existing item-ID route `POST/api/picking/item` remains separate.

Acceptance criteria and verification

Criterion	Expected result	Verification evidence (test methods)	Status
Barcode endpoint, valid pick	Barcode resolves to stock and the pick succeeds.	`PickingControllerTest.post_valid_barcode_picks_stock_and_returns_result`; `PickingServiceTest.picks_stock_resolved_from_a_barcode_and_decreases_quantity`.	Passed
Invalid request	HTTP 400 for quantity below one.	`PickingControllerTest.post_zero_quantity_returns_bad_request`; `PickingServiceTest.rejects_zero_and_negative_quantities`.	Passed
Unknown barcode	HTTP 404, no stock created.	`PickingControllerTest.post_unknown_barcode_returns_not_found`; `BarcodeResolverTest.rejects_an_unknown_barcode_without_creating_stock`.	Passed
Unavailable/insufficient stock	HTTP 409.	`PickingControllerTest.post_unavailable_stock_returns_conflict`, `post_excessive_quantity_returns_conflict`; `PickingServiceTest.rejects_unavailable_stock`, `rejects_quantity_greater_than_available_stock`.	Passed
Existing item-ID route	Remains separate and available.	`PickingControllerTest.direct_item_id_endpoint_remains_separate_and_available`; `PickingServiceTest.direct_item_id_picking_remains_available_separately`.	Passed

Verification note: the current source test total is 14 (6 controller, 2 resolver, 6 service), re-counted from the source in the re-audit. Stale ignored Surefire output for a missing integration-test source is excluded.

Before and after. Before commit: `5ca90cc12be1141be47cb6e6bdf3ca26efe636bd`. After commit: `0f0fbf0a3ca66c11f10cef127cbb2750a0ce53e8`. These commits belong to the original local project history, which was not copied into the team repository; only the after source tree is published there. The project is a standalone reference implementation, not a patch to the canonical OpenWMS Picking module.

Implementation and tests. Barcode resolution, pick-service behavior, and controller responses are separated into focused code and tests. Recorded `mvn.cmd -DforkCount=0 verify` completed with **BUILD SUCCESS**; 14/14 current source tests passed. The ignored generated `PickingControllerIT.xml/.txt` Surefire artifacts refer to a test source file that is absent; their reported six tests are stale and are explicitly excluded from the 14-test total.

Pattern effects. No new GoF pattern was confirmed. The MVC controller/service separation is continued; barcode lookup is a new use case, not sufficient by itself to claim Strategy, Adapter, or another GoF pattern.

Refactoring. The principal design improvement is separation of barcode resolution, pick valida-

tion/processing, and HTTP response mapping rather than embedding all behavior in one controller action. This is a reference implementation with in-memory demonstration storage, so the tests establish the behavior of this sample rather than production persistence or hardware integration.

Metric	Before	After
Production Java files	7	14
Test Java files	1	3
Total Java files	8	17
Java LOC	197	526
Test cases passed	1	14

The after values were re-measured from the published source and match. The added LOC reflect added functionality and tests; more LOC is not itself evidence of better design.

3.3 Automatic Stock Reordering — Automatic stock-reorder rules

Change and acceptance criteria. Reorder evaluation triggers when `availableQuantity<=reorderThreshold` and `availableQuantity<targetQuantity`. The requested quantity is `targetQuantity-availableQuantity`. Re-evaluation reuses an existing OPEN request. Completing a request replenishes stock and marks the request complete, allowing a later low-stock event to create a new request.

Acceptance criteria and verification

Criterion	Expected result	Verification evidence (test methods)	Status
Threshold condition	Triggers only at or below threshold and below target.	`ReorderRuleTest.stock_above_threshold_does_not_require_reorder`, `stock_equal_to_threshold_requires_reorder_to_target`; `ReorderServiceTest.stock_above_threshold_does_not_create_a_request`.	Passed
Quantity calculation	Requested = target − available.	`ReorderRuleTest.stock_below_threshold_calculates_quantity_to_target`; `ReorderServiceTest.stock_equal_to_threshold_creates_a_request_for_the_gap_to_target`, `stock_below_threshold_creates_a_request_for_the_gap_to_target`.	Passed
Rule validation	Negative threshold or target ≤ threshold rejected.	`ReorderRuleTest.rejects_negative_threshold`, `rejects_target_that_is_not_greater_than_threshold`; `StockReorderControllerTest.invalid_rule_is_rejected`.	Passed
Open-request reuse	No duplicate while OPEN.	`ReorderServiceTest.repeated_evaluation_reuses_the_open_request`.	Passed
Completion and later reorder	Stock replenished; a later low-stock event creates a new request.	`ReorderServiceTest.automatic_reorder_after_consumption_and_completion_allow_a_future_request`, `completing_an_already_completed_request_is_rejected`; `StockReorderControllerTest.completing_request_replenishes_stock_and_allows_a_later_reorder`.	Passed
Persistence and distributed concurrency	State survives restart; distributed evaluation is safe.	Not implemented; storage is in memory and `evaluate`/`complete` are `synchronized` within one process.	Not satisfied

Verification note: 19 test methods exist in the source (`ReorderRuleTest` 5, `ReorderServiceTest` 8, `StockReorderControllerTest` 4, `StockServiceTest` 2), matching the recorded 19/19. Further mappings for all eleven CH03 criteria are in `report/part2_findings.pdf`.

Before and after. Before commit: `e9ad4c171771f12865de763b7f99e909b2539c47`. After commit: `59b1b59ff6679d13d1c36781e2a76192bbb1a4c7`. As with Barcode-Based Picking, this history is not included in the team repository. This is a separate reference implementation, not a modification of the inaccessible canonical Inventory module.

Implementation and tests. The sample adds stock-rule and reorder-request operations, with

endpoints for reading/consuming stock, managing rules, evaluating reorders (204 when no reorder is needed), listing and retrieving requests, and completing requests. Recorded `mvn.cmd-DforkCount=0verify` completed with `BUILD SUCCESS`; 19/19 tests passed.

Pattern effects. No new GoF pattern was confirmed. The rule predicate does not by itself establish Strategy because the implementation does not demonstrate a family of interchangeable algorithms selected through a common strategy abstraction. Existing MVC and repository-style separation is extended.

Refactoring and limitations. The implementation separates stock, rule, and request responsibilities and makes repeated evaluation idempotent while a request remains OPEN. Storage is in memory, not durable; synchronization is only within the application process. It is not production-ready for concurrent distributed deployments without persistence and concurrency control.

Metric	Before	After
Production Java files	6	24
Test Java files	1	4
Total Java files	7	28
Java LOC	169	978
Test cases passed	2	19

The after values were re-measured from the published source and match.

3.4 Updated pattern counts and interpretation

Across the recorded changes, the confirmed GoF inventory remains Observer and Composite: 2 distinct types and 2 instances in the Part 1 baseline. **Barcode-Based Picking** and **Automatic Stock Reordering** explicitly introduce no new confirmed GoF instance. **Multiple-Warehouse Support** changes warehouse domain identity and lookup behavior but does not establish a new GoF collaboration. The confirmed baseline participant count remains the Part 1 count; no new GoF participants are added to that inventory by these evolution scenarios. The Part 1 inventory is intentionally not inflated by ordinary controller, service, entity, or repository classes.

Snapshot / change	Types	Instances	Concrete classes	Rejected candidates	Notes
Part 1 baseline	2	2	2	2	Observer and generic Composite confirmed.
After **Multiple-Warehouse Support**	2	2	2	2	No new GoF pattern confirmed; changes remain uncommitted.
After **Barcode-Based Picking**	2	2	2	2	Separate reference implementation; no new GoF pattern.
After **Automatic Stock Reordering**	2	2	2	2	Separate reference implementation; no new GoF pattern.

The counts describe the frozen Part 1 inventory carried forward for cross-part consistency, not an exhaustive new pattern scan of each reference project. Changes are valuable because they address the requirements and clarify responsibilities; raw LOC increases do not prove design quality.

4 Part 3 Cross-language comparison

4.1 Selected instances and common specifications

The selected patterns are the source-verified Observer and Composite concepts from Part 1. The four-language implementations are independent, small reference implementations used to compare expression of the same concepts; they do not modify or claim to reproduce the internal OpenWMS implementation.

Observer specification. Support subscribe, unsubscribe, and publish. With three observers registered, publish `STOCK_LOW`; unsubscribe observer 2; publish `STOCK_REPLENISHED`. The second publication must reach observers 1 and 3, but not observer 2.

Composite specification. Represent a hierarchy `Warehouse>Zone-A>A-01/A-02` and `Warehouse>Zone-B>B-01`. Nodes share a common interface for name, child addition/removal, child retrieval, and recursive preorder traversal. Unlike the OpenWMS `TreeNode`, these implementations use separate `Container` and `Leaf` classes and include a recursive `traverse` operation on the component.

4.2 Implementations and actual results

The source and executable test harnesses are grouped under `Part3-cross-language/observer/` and `Part3-cross-language/composite/`, with Java, Python, JavaScript, and C++ subdirectories. Toolchain versions are listed in the environment section and detailed in `Part3-cross-language/PART3_EVIDENCE.md`. All eight pattern/language combinations passed: Observer 3/3 and Composite 4/4 in each language, for 28/28 checks overall. The re-audit re-ran all eight harnesses on a second machine and toolchain and again obtained 28/28. Per-harness console log files were not retained in the published evidence tree; the recorded commands/results should not be misrepresented as attached raw logs.

4.3 Static metrics

The following LOC totals include implementation and test code. Maximum cyclomatic complexity is the recorded Lizard 1.24.1 result; complexity-parser semantics are tool-dependent. All values were recomputed in the re-audit and match.

Pattern	Language	Impl. LOC	Test LOC	Total LOC	Max CCN	Checks passed
Observer	Java	65	70	135	2	3/3
Observer	Python	52	37	89	3	3/3
Observer	JavaScript	54	43	97	2	3/3
Observer	C++	55	61	116	2	3/3
Composite	Java	89	63	152	2	4/4
Composite	Python	71	48	119	2	4/4
Composite	JavaScript	81	47	128	2	4/4
Composite	C++	85	62	147	2	4/4

The machine-readable values are in `Part3-cross-language/metrics/cross_language.csv` and `pattern_metrics.csv`. The recorded timing protocol used one warm-up and 30 timed process invocations; Java/C++ compilation was excluded. The workload is very small and process startup dominates, so the timing values do not support a language-performance ranking. The CSV is the source of record for any timing values; no timing value is inferred in this summary.

4.4 Comparison and technology discussion

Observer maps naturally to interfaces and listener collections in Java, ABC-based contracts and object lists in Python, base classes and arrays in JavaScript, and pure virtual classes with non-owning pointers in C++. The important design property is the ability to register and remove independent consumers without making the publisher encode each consumer's behavior. Composite benefits from a shared component abstraction and recursive composition: clients can traverse a container and its descendants uniformly. Java expresses the abstraction through interfaces/classes; Python and JavaScript can express the same contract with less ceremony; C++ expresses it through classes and explicit ownership (`std::shared_ptr` children). These are qualitative observations from the small examples, not general performance claims.

The source artifacts, metrics, and exact commands are documented in `PART3_EVIDENCE.md`.

5 LLM records and human verification

The evidence record is `LLM_RECORDS/LLM_RECORDS.md`, with `Part3-cross-language/metrics/llm_records.csv` as its machine-readable index. Eight LLM-related records are present, including the 11 October re-audit (AUDIT-01). Four exact prompts are available; four records have historical prompt/model/access-tier metadata that could not be recovered. The report preserves that limitation rather than inventing model names, dates, or outputs.

Area	Verification evidence	Remaining limitation
Part 1	Source-based mappings re-checked line by line against the baseline; copied sources byte-identical; rejected candidates, counts, and notes.	Library scope: no concrete `EventListener` and no `TreeNode` client; N03 AOP wiring not in the repository.
Multiple-Warehouse Support	Dated verification addendum; changed-file list re-derived against upstream; 18 named CH01 test methods.	Changes uncommitted; no after commit ID or LOC metrics; migration/backfill absent; duplicate-coordinate rejection not tested; Asciidoctor errors noted.
Barcode-Based Picking	Source tests by name, acceptance criteria, commit pair, before/after metrics; after metrics re-measured.	Commit history not in the team repository; stale Surefire output excluded; no production storage/hardware claim.
Automatic Stock Reordering	Source tests by name, acceptance criteria, commit pair, before/after metrics; after metrics re-measured.	Commit history not in the team repository; in-memory state and single-process synchronization only.
Part 3	Eight implementations, 28/28 checks (re-run), CSV metrics (recomputed), documented timing protocol.	Per-harness raw console logs were not retained; timing not re-measured.

Generated code was treated as a candidate contribution and checked against the source, tests, and documented acceptance criteria. LLM-generated statements were not accepted as evidence without verification. Maven builds could not be re-run in the re-audit, so their totals remain the team's recorded runs. Missing historical details remain marked as unavailable.

6 Findings, contributions and references

6.1 Findings

1. The OpenWMS CORE baseline supports two confirmed GoF pattern instances: an Observer event-dispatching collaboration, with the Subject split into registration and notification interfaces, and a transparent, single-class Composite tree utility.
2. Both instances are confirmed at library level: the inspected repository contains no concrete `EventListener` and no client of `TreeNode`.
3. The OpenWMS Observer matches subscribers by exact event class and notifies them synchronously; asynchrony comes from the separate publisher (N01).
4. The OpenWMS Composite has no recursive component operation; it is a Composite by structure and interface rather than by recursive behavior.
5. **Multiple-Warehouse Support** models warehouse-scoped location identity and passes its recorded verification, but the work remains uncommitted, duplicate-coordinate rejection is enforced by schema without a test, and production rollout needs a deployment-specific mapping/backfill plan.
6. **Barcode-Based Picking** demonstrates barcode-driven picking behavior with 14/14 tests in a standalone reference project; it is not a change to the canonical Picking module.
7. **Automatic Stock Reordering** demonstrates threshold/target-based automatic reorder behavior with 19/19 tests in a standalone reference project; its in-memory state and single-process synchronization are not production persistence/concurrency controls.
8. Part 3 demonstrates consistent Observer and Composite behavior in Java, Python, JavaScript, and C++, with 28/28 checks reproduced on two toolchains and recorded static metrics. Its tiny timing workload does not justify performance conclusions.
9. The evidence shows requirement coverage and separation of responsibilities, but neither LOC growth nor passing tests alone establishes that a design is universally better.

6.2 Team contributions

The final contribution statements must be reviewed and confirmed by all team members. The current record intentionally does not infer individual work allocation from the roster alone.

Roll number	Name	Batch	Contribution
AM.SC.U4CSE23215	D Aditya Kiran	CSE C	To be confirmed by the team against actual work and commits.
AM.SC.U4CSE23235	Mithun Krishna	CSE C	To be confirmed by the team against actual work and commits.
AM.SC.U4CSE23248	Sai Kishen K M	CSE C	To be confirmed by the team against actual work and commits.
AM.SC.U4CSE23359	Sivadev Smijith	CSE D	To be confirmed by the team against actual work and commits.

6.3 References and evidence artifacts

1. OpenWMS CORE repository: https://github.com/openwms/org.openwms, baseline `9d263bf65ec75301e0302539ae7bf581412742b8`; source of the Part 1 classes and of the README statements on architecture and module accessibility.
2. OpenWMS Common Service Library: https://github.com/openwms/org.openwms.common.service.lib, baseline `a6566d7a90364f8cb0e82ec013720b644926dffe`.
3. OpenWMS Common Service: https://github.com/openwms/org.openwms.common.service,

baseline `fc7edd4ffbe16de65d482b3a2d462d4d896b1998`.

4. Part 1 findings and counts: `Part1_Findings/Part1_Findings.md`; `Part1_Findings/data/patterns.csv`; `Part1_Findings/data/pattern_counts.csv`; `Part1_Findings/data/rejected_candidates.csv`; `Part1_Findings/evidence/verification_notes.md`.
5. Part 1 diagrams: `observer_uml.puml`, `composite_uml.puml`, and `architecture_scope.puml`, with rendered PNGs, in `Part1_Findings/diagrams/`.
6. **Multiple-Warehouse Support** provenance and verification: `Part2/CH01-multiple-warehouses/PROVENANCE.md`; `LLM_RECORDS/LLM_RECORDS.md`, including the dated verification addendum.
7. **Barcode-Based Picking** evidence and source: `Part2/CH02-barcode-picking/CH02_EVIDENCE.md`; before commit `5ca90cc12be1141be47cb6e6bdf3ca26efe636bd`; after commit `0f0fbf0a3ca66c11f10cef127cbb2750a0ce53e8`.
8. **Automatic Stock Reordering** evidence and source: `Part2/CH03-stock-reorder/CH03_EVIDENCE.md`; before commit `e9ad4c171771f12865de763b7f99e909b2539c47`; after commit `59b1b59ff6679d13d1c36781e2a76192bbb1a4c7`.
9. Part 3 implementations, metrics, and evidence: `Part3-cross-language/PART3_EVIDENCE.md`; `Part3-cross-language/metrics/cross_language.csv`; `Part3-cross-language/metrics/pattern_metrics.csv`; `Part3-cross-language/metrics/llm_records.csv`.
10. Complexity tool: Lizard 1.24.1, recorded in the Part 3 evidence. Compiler/runtime versions are listed in the same evidence.
11. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 (Observer, Composite, Singleton, Proxy).

7 Team folder and datasets

The team working repository keeps the report, raw inventories, change-case evidence, tests, and cross-language metrics traceable. Its current organization is:

```
Team_B9_DP/
  README.md
  report/
    main.tex, main.pdf
    part1_findings.tex/.pdf, part2_findings.tex/.pdf, part3_findings.tex/.pdf
  Aditya_findings/
    README.md
    COPY_MANIFEST.md
    LLM_RECORDS/LLM_RECORDS.md
    Part1_Findings/
      Part1_Findings.md
      data/  diagrams/  evidence/
    Part2/
      CH01-multiple-warehouses/
      CH02-barcode-picking/
      CH03-stock-reorder/
    Part3-cross-language/
      observer/  composite/  metrics/
      PART3_EVIDENCE.md
```

The CH02 and CH03 Git histories (and therefore their before commits) and the CH01 upstream worktrees are not part of this repository; upstream OpenWMS source is included only as the evidence excerpts and snapshots described in `COPY_MANIFEST.md`, with Apache 2.0 notices preserved.

The repository screenshot required by the assignment should be captured from the final published submission repository.